

Les nombres binaires

Notes de cours rédigées à l'aide de Gemini

L'être humain utilise naturellement le [système décimal](#), basé sur le nombre 10. Ce choix vient simplement de notre corps : nous avons dix doigts sur nos deux mains pour compter.

Origine et usage

- Comptage : Nos ancêtres ont utilisé les dix doigts comme des unités de base.
- Vie courante : Le calcul de tous les jours emploie dix chiffres (de 0 à 9).

Autres systèmes dans l'histoire

- Base 20 : Certains peuples utilisaient aussi les orteils pour compter jusqu'à vingt.
- Base 60 : Les anciens Babyloniens utilisaient une base de 60, visible dans nos minutes et nos heures.

Vous avez probablement entendu que les ordinateurs utilisent le système binaire basé sur le nombre 2.

Pourquoi?

L'ordinateur utilise le système binaire (composé de 0 et de 1) parce qu'il fonctionne avec des composants électroniques, comme les transistors, qui agissent de manière fiable comme de simples interrupteurs fermés (courant coupé, 0) ou ouverts (courant passant, 1). C'est un peu comme un interrupteur qui contrôle une lumière 💡.

Comment fonctionne le système binaire?

Si je vous dis le nombre 3287, vous savez que c'est l'équivalent de

$$3 \times 1000 + 2 \times 100 + 8 \times 10 + 7 \times 1$$

ou

$$3 \times 10^3 + 2 \times 10^2 + 8 \times 10^1 + 7 \times 10^0$$

Il y a 10 chiffres possibles pour représenter les unités (0, ..., 9).

Si on ajoute 1 au nombre 9, il faut maintenant représenter ce nouveau nombre par 10.

Puis, 11, 12, ..., 19 et finalement, $19 + 1$ est 20, etc.

Combien de nombres peut-on représenter avec une seule main? 5? En fait, on peut représenter 32 nombres avec une seule main.



<https://x.com/DrEugeniaCheng/status/635500842867630080>

On peut représenter de $[0, 31]$ avec 5 doigts.

Dans le vocabulaire informatique, un chiffre binaire (0 ou 1) s'appelle un bit (binary digit). En anglais, digit veut dire chiffre mais aussi doigt.

Introduction à la programmation

420-SF1-RE

Les nombres binaires(Automne 2026)

Version 1.0

Système décimal	Système binaire	doigts de la main
0	0	
1	1	
2	10	
3	11	
4	100	
5	101	
6	110	
...	...	...
31	11111	

11111_2 est égale à $1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 16 + 8 + 4 + 2 + 1 = 31$.

L'indice $_2$ dans 11111_2 indique qu'on utilise la base 2.

Souvent, on va regrouper 8 bits ensemble. On appelle cela un octet (byte en anglais).

Donc 00011111_2

123_{10} est égale à

$$0 \times 128 + 1 \times 64 + 1 \times 32 + 1 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 01111011_2$$

Si on travaillait avec la base 8, ce serait le même principe

$$123_{10} = 173_8 = 1 \times 8^2 + 7 \times 8^1 + 3 \times 8^0 = 64 + 56 + 3$$

Si on travaillait en base 6 😊 (non, personne n'utilise la base 6, ce n'est qu'un exemple), on aurait

$$3 \times 6^2 + 2 \times 6^1 + 3 \times 6^0 = 3 \times 36 + 2 \times 6 + 3 \times 1 = 108 + 12 + 3 = 123$$

C'est comme cela qu'on représente des entiers en binaire.

Les bits se trouvent dans la mémoire de l'ordinateur.

Travailler avec des 0 et des 1 est difficile. Les programmeurs utilisent la base 16 ou le système hexadécimal. On regroupe 4 bits qui peuvent représenter $[0, 15]$. Les chiffres de la base 16 sont 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

$$A_{16} = 10_{10}$$

$$B_{16} = 11_{10}$$

$$F_{16} = 15_{10}$$

$$0x7B \text{ représente donc } 7 \times 16^1 + 11 \times 16^0 = 112 + 11 = 123$$

Les caractères

Si vous ouvrez un éditeur de texte sur votre ordinateur et que vous tapez *Bonjour* dans l'éditeur. Comment sont représentés les caractères B, o, n, j, o, u, r? En jargon informatique, on appelle cela des caractères. Les lettres de l'alphabet sont un sous-ensemble des caractères. `!@#$` sont aussi des caractères.

Ils sont aussi représentés avec des 0 et des 1.

ASCII (*American Standard Code for Information Interchange*) est un code informatique qui permet de traduire les lettres, les chiffres et les signes de ponctuation en nombres. Ces nombres permettent aux ordinateurs, qui ne comprennent que les zéros et les uns, de stocker et d'afficher du texte.

Il y a une convention où on a assigné des valeurs numériques aux caractères.

Dec	Chr	Dec	Chr	Dec	Chr	Dec	Chr	Dec	Chr
0	NUL	26	SUB	52	4	78	N	104	h
1	SOH	27	ESC	53	5	79	O	105	i
2	STX	28	FS	54	6	80	P	106	j
3	ETX	29	GS	55	7	81	Q	107	k
4	EOT	30	RS	56	8	82	R	108	l
5	ENQ	31	US	57	9	83	S	109	m
6	ACK	32		58	:	84	T	110	n
7	BEL	33	!	59	;	85	U	111	o
8	BS	34	"	60	<	86	V	112	p
9	HT	35	#	61	=	87	W	113	q
10	LF	36	\$	62	>	88	X	114	r
11	VT	37	%	63	?	89	Y	115	s
12	FF	38	&	64	@	90	Z	116	t
13	CR	39	'	65	A	91	[	117	u
14	SO	40	(	66	B	92	\	118	v
15	SI	41	)	67	C	93	]	119	w
16	DLE	42	*	68	D	94	^	120	x
17	DC1	43	+	69	E	95	_	121	y
18	DC2	44	,	70	F	96	`	122	z
19	DC3	45	-	71	G	97	a	123	{
20	DC4	46	.	72	H	98	b	124	
21	NAK	47	/	73	I	99	c	125	}
22	SYN	48	0	74	J	100	d	126	~
23	ETB	49	1	75	K	101	e	127	DEL
24	CAN	50	2	76	L	102	f		
25	EM	51	3	77	M	103	g		

<https://www.lookuptables.com/text/ascii-table>

Bonjour est représenté par les valeurs 66, 111, 110, 106, 111, 117 et 114.

Quand vous tapez le mot *Bonjour*, l'éditeur de texte dit à l'ordinateur que le nombre 66 n'est pas un entier mais qu'il doit se référer au tableau pour savoir quel caractère il doit afficher.

C'est vraiment le programme (le code qu'on écrit) qui indique à l'ordinateur si la série de 0 et de 1 représente un entier ou un caractère (ou d'autre chose).

L'ASCII est limité à 128 codes. Il a été remplacé par l'Unicode qui peut représenter jusqu'à 2^{16} codes. L'Unicode peut représenter les caractères de toutes les langues vivantes (et mortes) et même les [émoticons](#).

Emoticons (U+1F600 to U+1F64F)

											
128512	128513	128514	128515	128516	128517	128518	128519	128520	128521	128522	128523
											
128524	128525	128526	128527	128528	128529	128530	128531	128532	128533	128534	128535
											
128536	128537	128538	128539	128540	128541	128542	128543	128544	128545	128546	128547
											
128548	128549	128550	128551	128552	128553	128554	128555	128556	128557	128558	128559
											
128560	128561	128562	128563	128564	128565	128566	128567	128568	128569	128570	128571
											
128572	128573	128574	128575	128576	128577	128578	128579	128580	128581	128582	128583
											
128584	128585	128586	128587	128588	128589	128590	128591				